On-Device Classification Using Real Light Sensor Data

Ali Hussain

1 Proposed Pipeline

The classification pipeline is designed so that the expensive training steps run offline on a computer, while the ESP32 performs only a small lookup during live operation.

Two light sensor values are collected from the hardware. An exponential moving average (EMA) filter is applied to reduce short transient changes in the sensor readings. Because the sunlight measurements have a much larger numeric range than the indoor measurements, the two sensor values are transformed using

$$x' = \log(1 + x).$$

The transformed values are then scaled using min–max bounds calculated from the training data.

DBSCAN is used offline to discover groups in the two-dimensional sensor feature space without using the known class labels. A Gaussian kernel density estimate (KDE) is then fitted to each discovered class. Each KDE surface is normalized, sampled onto a fixed grid, and converted to `uint8` so that every grid cell uses one byte.

The ESP32 does not run DBSCAN or KDE. It stores the generated byte tables, scaling bounds, class names, grid size, and reject threshold. During live inference it filters the sensor readings, applies the same logarithmic transformation and scaling, finds the appropriate grid cell, reads one byte from each class table, and selects the class with the highest stored value.

Algorithm 1 On-device table lookup classification

Require: Raw sensor readings (x, y)
Require: EMA filter state
Require: One byte table Q_c for each class
Require: Training scaling bounds
Require: Grid size G
Require: Reject threshold τ
Ensure: Predicted lighting class or UNKNOWN

Apply EMA filter to x and y
$x \leftarrow \log(1 + x)$
$y \leftarrow \log(1 + y)$
Scale x and y using stored training bounds
$col \leftarrow \lfloor \tilde{x} G \rfloor$
$row \leftarrow \lfloor \tilde{y} G \rfloor$
Clip row and col to $[0, G - 1]$
for each class c **do**
    $v_c \leftarrow Q_c[row, col]/255$
end for
$\hat{c} \leftarrow \arg\max_c v_c$
if $v_{\hat{c}} < \tau$ **then**
    return UNKNOWN
end if
return CLASS_NAMES$[\hat{c}]$

2 Application and Hardware

The proof-of-concept system classifies lighting conditions using two sensor measurements connected to an ESP32.

The first input is a UV sensor connected to an ESP32 analog input. The second input is the infrared channel of a TSL2591 light sensor connected through I2C. The hardware used for the experiment therefore differs from the original analog IR sensor circuit, but it still provides two light-related sensor features for classification.

Four repeatable lighting conditions were collected:

- shade,
- LED,
- fluorescent,
- sunlight.

The ESP32 collection firmware sampled the sensors at approximately 10 Hz and logged the two filtered sensor values together with the current class label.

Table 1: Number of collected samples per lighting class.

Class	Samples
Shade	600
LED	600
Fluorescent	600
Sun	600

An exponential moving average was used with

$$\alpha = 0.2.$$

The filter follows

$$y_n = \alpha x_n + (1 - \alpha) y_{n-1}.$$

This filter was chosen because it reduces short sensor fluctuations while remaining simple enough to run continuously on the ESP32.

3 Method and Setup

3.1 Real Dataset

The real dataset contains 2400 labelled sensor samples. Each lighting class contains 600 samples:

The two features used for clustering were the UV sensor reading and the IR sensor reading. The known lighting label was kept separate and was not supplied to DBSCAN.

The dataset was divided into:

- 60% training,
- 20% validation,
- 20% test.

This produced:

1440 training samples,

480 validation samples,

and

480 test samples.

The test set is kept separate from parameter selection and will be used later for the final desktop and on-device evaluation.

Table 2: Ground-truth label versus DBSCAN cluster on the training split.

True label	C0	C1	C2	C3
LED	360	0	0	0
Fluorescent	0	360	0	0
Shade	0	0	360	0
Sun	0	0	0	360

3.2 Preprocessing

Initial DBSCAN experiments showed that the raw sensor ranges were very different. Sunlight produced much larger UV and IR readings than the indoor lighting conditions. After normal min–max scaling, the indoor classes were compressed close together.

To reduce this large numeric range, each sensor value was transformed using

$$x' = \log(1 + x).$$

The transformed training values were then min–max scaled. The scaling bounds were calculated using the training split only. The same stored bounds are used for validation, test, and later on-device inference.

3.3 DBSCAN

DBSCAN was re-tuned using the real sensor data. The final parameters were

$$\varepsilon = 0.02$$

and

$$\texttt{min_samples} = 8.$$

When DBSCAN was run on the 1440-point training split, it discovered four clusters with no noise points.

Each discovered cluster contained 360 training samples and corresponded to one of the four known lighting conditions.

The cluster numbers themselves are arbitrary. In this run, cluster 0 corresponded to LED, cluster 1 to fluorescent, cluster 2 to shade, and cluster 3 to sunlight.

Table 3: KDE bandwidth validation results.

Bandwidth h	Mean log-likelihood
0.03	5.157
0.04	4.660
0.05	4.317
0.06	4.049
0.08	3.626
0.10	3.285
0.12	2.994

3.4 KDE Bandwidth Selection

A Gaussian KDE was fitted separately to each discovered training cluster.

The KDE bandwidth was selected using the validation split. The tested values and mean validation log-likelihood are shown in Table 3.

The highest validation score was obtained with

$$h = 0.03,$$

so this value was selected for the real-data model.

3.5 Byte-Table Storage

Each normalized KDE surface was sampled onto a

$$16 \times 16$$

grid and converted to `uint8`.

There are four discovered classes, so the lookup-table storage is

$$KG^2 = 4 \times 16^2 = 1024 \text{ bytes}.$$

Each class requires

$$16^2 = 256 \text{ bytes}.$$

The four stored scaling bounds require another 16 bytes when represented as four `float32` values.

Therefore, the current predicted model storage is

$$1024 + 16 = 1040 \text{ bytes},$$

not including the remaining firmware instructions and class-name strings.

The worst-case measured error caused by converting the normalized KDE values to `uint8` was

$$0.193\%.$$

This indicates that the byte representation greatly reduces storage while introducing only a small quantization error.

4 Evidence

4.1 DBSCAN Compared With Ground Truth

The known labels were not supplied to DBSCAN. They were used only after clustering to evaluate the discovered groups.

Before the logarithmic transformation, DBSCAN tended to merge shade, LED, and fluorescent into the same cluster while splitting the sunlight measurements into multiple clusters.

The sensor plot showed why this happened. Sunlight varied strongly in both sensor dimensions, while the indoor classes had very small UV readings and were separated mainly by the IR sensor.

After applying the logarithmic transform and rescaling the two features, DBSCAN produced four stable clusters. With $\varepsilon = 0.02$ and `min_samples=8`, all 1440 training points were assigned to the correct discovered group with no noise points.

The confusion table in Table 2 therefore shows complete agreement between the discovered training clusters and the known lighting conditions.

4.2 Validation Evidence

The KDE bandwidth was selected using validation data instead of selecting a value from the training fit.

Among the tested values, $h = 0.03$ achieved the highest mean validation log-likelihood of 5.157. The validation score decreased as the bandwidth was increased, so $h = 0.03$ was kept for the stored model.

4.3 Predicted Embedded Footprint

The current stored model contains four 16×16 byte tables.

The predicted lookup-table footprint is 1024 bytes, and the four scaling bounds add another 16 bytes. The predicted tables and bounds therefore occupy 1040 bytes.

The generated model will later be compiled into the ESP32 firmware. The measured flash usage, SRAM usage, and inference latency will be compared against this predicted storage requirement during on-device testing.

4.4 Filtering Evidence

The collection firmware uses an EMA filter with $\alpha = 0.2$ to reduce short-duration sensor fluctuations.

A raw-versus-filtered time-domain plot will be added after collecting the required raw and filtered readings from the hardware. This plot will be used to verify that the filter removes transient glitches without removing the differences between lighting classes.

4.5 On-Device Evaluation

The final ESP32 implementation will be tested using a fresh held-out batch of sensor measurements. The output of the embedded lookup classifier will be compared with the desktop implementation.

The final version of this section will report:

- on-device versus desktop agreement,
- measured flash usage,
- measured SRAM usage,
- inference time per query,
- comparison between predicted and measured cost.

4.6 Proof of Concept

A photograph of the final wired ESP32 system and a link to a 30–60 second video of live classification will be added after the embedded implementation is complete.

5 Limitations and Remaining Uncertainty

The current real dataset was collected under four controlled lighting conditions. Although each class contains 600 samples, the dataset does not yet represent every lighting condition that the device could encounter in normal use.

The indoor lighting classes are separated mainly by the IR feature because their UV readings remain close to zero. Sunlight contributes strongly to both sensor dimensions. This means that the two-dimensional feature space is not equally informative for every class.

A logarithmic transformation was added because the sunlight readings were much larger than the indoor readings. This improved DBSCAN clustering considerably, but the same transformation must also be reproduced on the ESP32. Any difference between desktop and embedded preprocessing could change the selected lookup cell.

The DBSCAN comparison currently shows perfect agreement on the training split, but this does not yet demonstrate final classifier accuracy. KDE and the stored byte tables still need to be evaluated using the held-out test data and then compared with the real ESP32 implementation.

The current model footprint of 1040 bytes includes the class lookup tables and scaling bounds only. The final firmware will use additional flash and SRAM for the program, sensor libraries, filter variables, class names, and other runtime data.

The reject threshold also still needs to be evaluated using the real held-out data. The final value should prevent low-density readings from being forced into one of the four known lighting classes without rejecting too many valid samples.

The next measurements will therefore focus on held-out classification agreement, reject behaviour, actual ESP32 flash and SRAM usage, inference latency, and the raw-versus-filtered sensor response.